

痩せない曲げ方はあった — Dual Quaternion で体積の減りが 154分の1 になる

2026-09-14 | Houdini 21.0.700 | /obj/rigskin | ノード 7 / 接続 8

何を作ったか

実験052で、骨を曲げると皮が痩せることを測った。90度で 27.13%、減り方は $27.1284\% \times (1 - \cos \theta)$ というきれいな式に乗っていた。

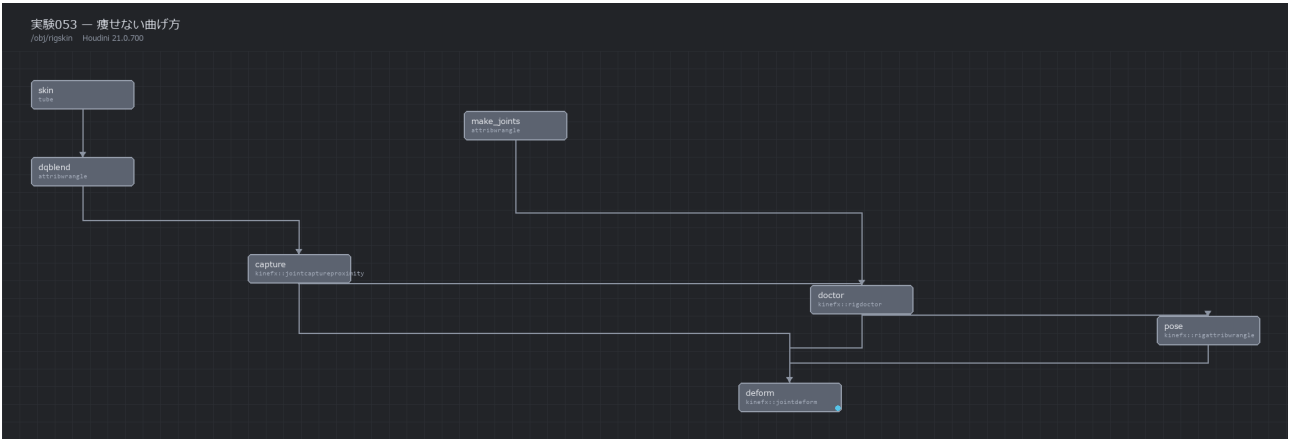
kinefx::jointdeform には混ぜ方が3つある。Linear（既定）、Dual Quaternion、その2つの中間。同じ骨・同じ皮・同じ角度で比べた。

90度での体積の減りは、Linear 27.1284%、Dual Quaternion 0.1764%、中間（半々）14.6904%。Dual Quaternion は Linear の 154分の1 しか痩せない。

そして式のほうも違った。減った分を $(1 - \cos \theta)$ で割ると、Linear は 6通りすべてで 27.1284~27.1286（幅 0.0001）とびたり一定。Dual Quaternion は 0.1513~0.2003（幅 0.0490）でそろわない。中間は 13.6663~15.6573（幅 1.9911）。式に乘るのは Linear だけだった。

「2つの中間」は、そのままでは動かない。点ごとの混ぜ具合を表す属性が要る（Missing dual quaternion blend attribute）。ここでは全点に 0.5 を入れた。すると減りは 14.6904% で、Linear の 27.1284% のほぼ半分。混ぜた割合がそのまま結果に出ている。

ノードの接続



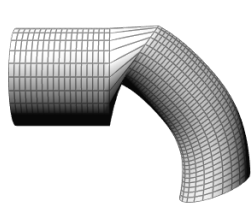
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
make_joints	attribwrangle	class = 0 snippet = int prim = addprim(0, "polyline"); for (int i = 0; i < 3; i++) { int pt = addpoint(0, set(0.0, float(i), 0.0)); ...
doctor	kinefx::rigdoctor	inittransforms = 1 outputparentidx = 1 transformations = 1
pose	kinefx::rigattribwrangle	angle = 90.0 snippet = if (s@name == "joint1") { matrix m = 4@localtransform; prerotate(m, radians(ch("angle"))), set(0, 0, 1)); ...

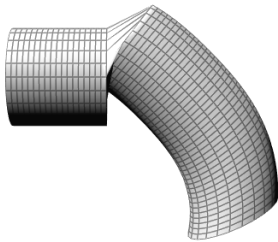
ノード	タイプ	デフォルトから変更したパラメータ
skin	tube	cap = 1 cols = 24 height = 2.0 rad1 = 0.32 rad2 = 0.32 rows = 60 ty = 1.0 type = 1
dqblend	attribwrangle	snippet = f@dqblend = 0.5;
capture	kinefx::jointcaptureproximity	(変更なし)
deform	kinefx::jointdeform	method = 1

同じ90度を、2つのやり方で

皮は 1,440点 / 1,418面 の筒。骨は関節3つ。カメラは2枚とも同じ。



Houdini



Houdini

Linear（既定）。内側が食い込み、体積は 27.13% 減る。Dual Quaternion。太さが保たれ、体積の減りは 0.18%。

角度	Linear（既定）	Dual Quaternion	2つの中間
0°	0.0000%	0.0000%	0.0000%
15°	0.9244%	0.0052%	0.4657%
30°	3.6345%	0.0205%	1.8417%
45°	7.9457%	0.0459%	4.0666%
60°	13.5642%	0.0808%	7.0413%
90°	27.1284%	0.1764%	14.6904%
120°	40.6926%	0.3005%	23.4860%

それぞれ式に乗るか（減った分 ÷ (1 - cos θ))

一定なら、その形の式に乗っている。

角度	Linear（既定）	Dual Quaternion	2つの中間
15°	27.1286	0.1513	13.6663

角度	Linear（既定）	Dual Quaternion	2つの中間
30°	27.1284	0.1532	13.7469
45°	27.1284	0.1568	13.8844
60°	27.1284	0.1616	14.0827
90°	27.1284	0.1764	14.6904
120°	27.1284	0.2003	15.6573
ばらつきの幅	0.0001	0.0490	1.9911

中心からの太さ（90度するとき、高さの帯ごと）

その帯の点が、帯の中心からどれだけ離れているかの平均。曲げる前の筒の半径は 0.32。

元の高さ	Linear（既定）	Dual Quaternion	2つの中間
0.00～0.50	0.305559	0.353538	0.329018
0.50～0.75	0.281015	0.329243	0.304530
0.75～1.00	0.279344	0.327047	0.302601
1.00～1.25	0.279592	0.327048	0.302731
1.25～1.50	0.328019	0.341245	0.333388
1.50～2.01	0.350916	0.350916	0.350916

わかったこと

- ・ Dual Quaternion は 154分の1 しか痩せない。90度で Linear が 27.1284% 減るのに対し、0.1764%。 $27.1284 \div 0.1764 = 153.8$ 倍の差。
- ・ 式に乗るのは Linear だけだった。(1 - cos θ) で割った値の幅は、Linear が 0.0001、Dual Quaternion が 0.0490、中間が 1.9911。Linear は式で言い切れるが、Dual Quaternion はこの形では表せない。
- ・ Dual Quaternion も、角度を上げると少しずつ痩せる。0.0052%（15度）→ 0.3005%（120度）。ゼロではない。ただし 120度でも 0.3% なので、実用上は無視できる。
- ・ 「2つの中間」は、そのままでは動かない。「Missing dual quaternion blend attribute」で止まる。点ごとの混ぜ具合を表す属性を、自分で作って名前を指定する。ここでは全点に 0.5 を入れた。
- ・ 半々に混ぜると、減りもほぼ半分になる。14.6904% は Linear の 27.1284% の 0.5415倍。混ぜた割合がそのまま結果に出る。部位ごとに混ぜ方を変えられるのが、この方式の使いどころになる。
- ・ 太さで見ても差ははっきり出る。関節の近く（高さ 0.75～1.00）で、Linear が 0.279344、Dual Quaternion が 0.327047。元の半径 0.32 に対して、Linear は 13% 細く、Dual Quaternion はほぼそのまま。
- ・ 関節から離れた場所では3つとも同じ。高さ 1.50～2.01 の帯はどのやり方でも 0.350916 で完全に一致する。骨1本にしか従っていない場所では、混ぜ方は関係ない。この一致は、測り方が正しいことの確認にもなっている。
- ・ ではなぜ Linear が既定なのか。この実験からは分からない。速さか、ねじれの扱いか、互換性か。確かめていないので断定しない。次に測るとすれば、そこになる。

次にやること

- Linear と Dual Quaternion の速さを比べる
- ねじれ（ひねり）を入れたときの違いを測る
- 関節の回る範囲を決める（`kinefx::configurejointlimits`）